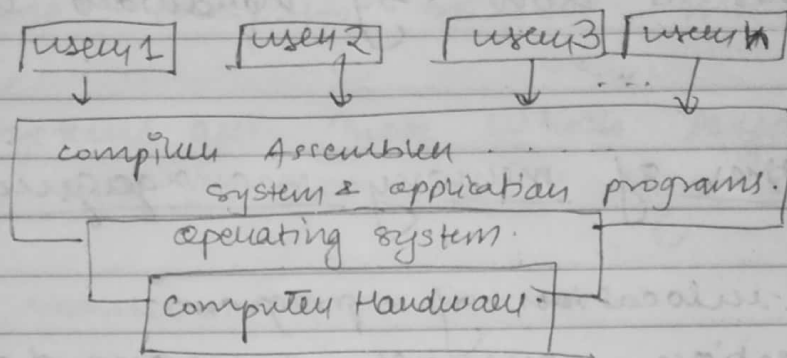


Abstract view of system.

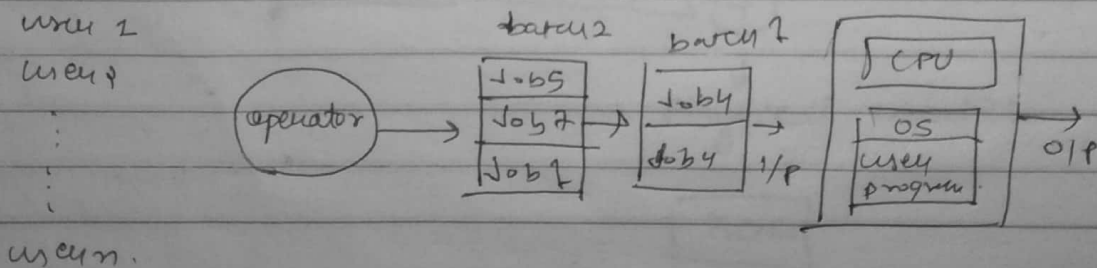


Job

Program.
I/p data
Control instructions.

Batch processing :-

→ Jobs with similar needs are batched together and executed through the processor as a group.



→ operator sort jobs as a deck of punch cards into batch of with similar needs.

Advantages

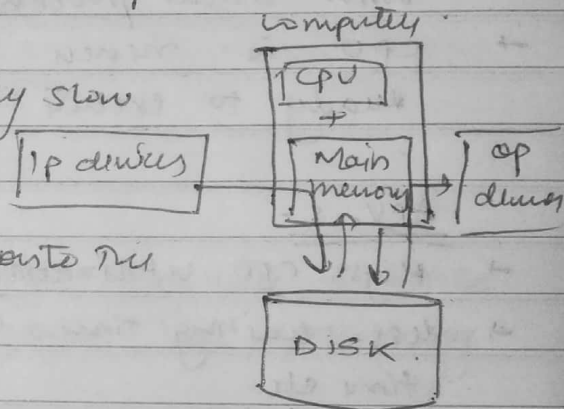
- In batch job execute one after another saving time from activities like loading compiler.
- during a batch execution no natural intervention is needed.

Disadvantages.

- memory limitation.
- interaction of I/P O/P directly with CPU.

Spooling: Simultaneous peripheral operations online.

- Input output devices are relatively slow as compared to CPU (digital)



- In spooling data is stored first onto the disk via m.m.

- Spooling is capable of overlapping I/O operations for one job with CPU operations of other jobs

Advantages :-

- no interaction of I/P & O/P devices with CPU.
- CPU utilization is more as CPU is busy most of the time.

Disadvantages:

- In starting spooling was unprogramming.

MULTIPROGRAMMING OPERATING :-

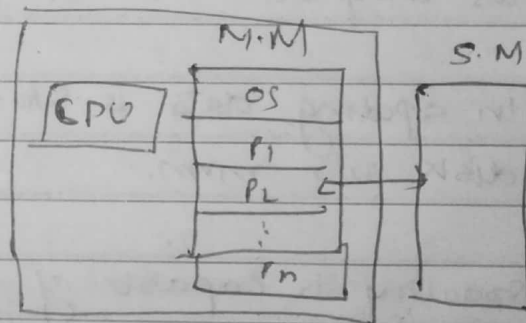
- maximise CPU utilization.
- Multiprogramming means more than 1 process in main memory which are ready to execute.
- Process generally require CPU time and I/O time, so if running process performs I/O or some other event which do not require CPU then instead of sitting idle CPU make a context switch and picks some other process and this idea will continue.
- CPU is never idle unless there is no process ready to execute or at time of context switch.

Adv :-

- High CPU utilization.
- less waiting time, response time etc.
- May be extended to multiple users.
- Now a days useful when load is more (very high).

Dis Adv :-

- Difficult scheduling.
- Main Memory management is required.
- Memory fragmentation.
- Paging (Non contiguous memory allocation.)



Multitasking / Time sharing :-

(Fair share, multiprog. with time sharing)

- Multitasking is multiprogramming with time sharing
- only one CPU but switches between ^{so quickly} ~~all~~ ^{all} executing that it creates an illusion that all ^{are} executing at the same time.
- the task in multitasking may refer to multiple threads of the same program.
- Main idea is better response time and executing multiple process together.

Multiprocessing :-

- two or more CPU with a single computer in close communication sharing the system bus, memory & other I/O devices.
- Different process may run on different CPU, each CPU has equal rights.
- Symmetric : one OS control all CPU & each CPU has equal rights.
- Asymmetric : Master slave architecture, system task on one processor and application on another one CPU will handle all hardware interrupt or I/O devices they are easy to design but less efficient.

Advantage

- Increased throughput
- Increased Reliability
- Cost saving
- battery efficiency
- true parallel processing.

Disadvantage

- more complex
- Overhead or coupling reduces throughput
- large main memory

CPU scheduling :-

- A process execution consists of a cycle of CPU execution and I/O execution.
- Normally every process ~~will~~ begins with CPU burst then I/O burst then CPU then I/O & so on and will end up on CPU burst.

CPU bound process : Those processes which requires most ^{time on} of ~~the~~ CPU. ~~time~~

I/O bound process : - most ~~of~~ the time required is of I/O

CPU

Non-pre-emptive

Pre-emptive

- when a process complete its execution
- when a process leaves CPU voluntarily to perform some I/O operation or to wait for an event

- Burst time/execution/running : It is the time process requires for running.
- waiting time : the time spent by a process in ready state waiting for CPU.
- Arrival time : when a process enters ready state.
- exit time : when process completes execution and exit from system.
- Turnaround time : Total time spend by a process in system

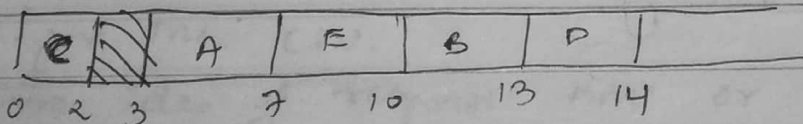
$$T.A.T = E.T - A.T = B.O.T + W.O.T$$
- Response time : Time between a process enters ready state/queue and get scheduled on CPU for the first time.

FCFS :-

- simplest scheduling algo. It assigns CPU to the process in order of arrival first.

- queue.
- Always non-preemptive.

Gantt chart



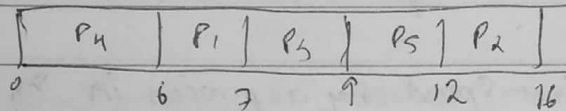
P.ID	AT	BT	T.T
A	3	4	$7-3=4$
B	5	3	$13-5=8$
C	0	2	$3-0=3$
D	5	1	$14-5=9$
E	4	3	$10-4=6$

convoy effect : smaller process have to wait for long time bigger process to release CPU.

SJFS :-

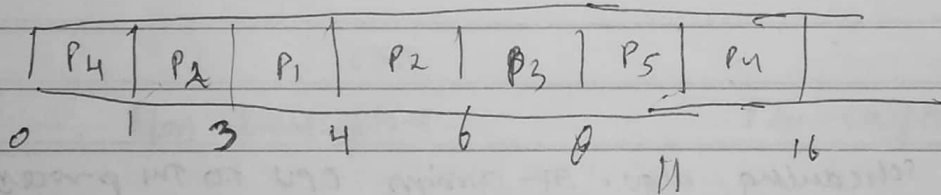
(non preemptive)

→ CPU is assigned to the process with the smallest burst time.



P.T	A.T	B.T
P ₁	3	1
P ₂	1	13
P ₃	4	2
P ₄	0	6
P ₅	2	3

Example (SRTF) :- minimal average waiting time.



Disadv :-

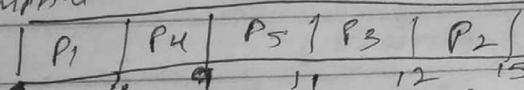
- No way to implement the algo.
- Process with longer CPU burst time will go into starvation.
- No idea of priority.

PSA :-

- Priority is associated with each process.

P_id	A.T	B.T	Priority	TAT	TWT
P ₁	0	43	2		
P ₂	1	32	3		
P ₃	2	1	4		
P ₄	3	5	5		
P ₅	4	2	5		

non pre-emptive

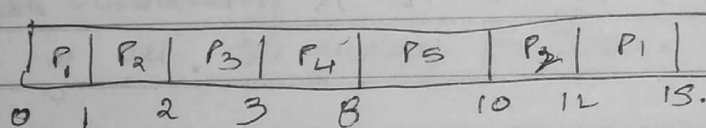


→ Tie is broken using FCFS order.

→ no importance to A.T or B.T

→ supports both non pre-emptive and pre-emptive approaches.

Pre-emptive



Adv :-

- facility of priority for system process.
- allows to run imp't process first.

Disadv :-

- Process with the smaller priority may starve for the CPU.
- no idea of response time or W.T

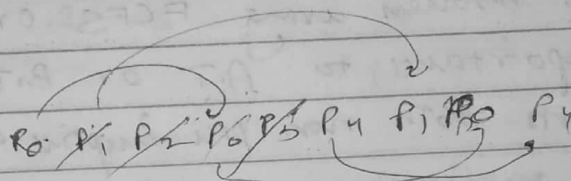
Round Robin :-

This process is designed for time sharing system where it is not necessary to complete any process and then start another but to be responsive.

and divide time of the CPU among the process in ready state.

- Ready queue will be treated as circular queue.

Pid	ArrT	B.T
P ₀	0	5/3/2
P ₁	1	3/2
P ₂	2	1
P ₃	3	2
P ₄	4	3/2



P ₀	P ₁	P ₂	P ₀	P ₃	P ₄	P ₁	P ₀	P ₄	
0	2	4	5	7	9	11	12	13	14

ADV :-

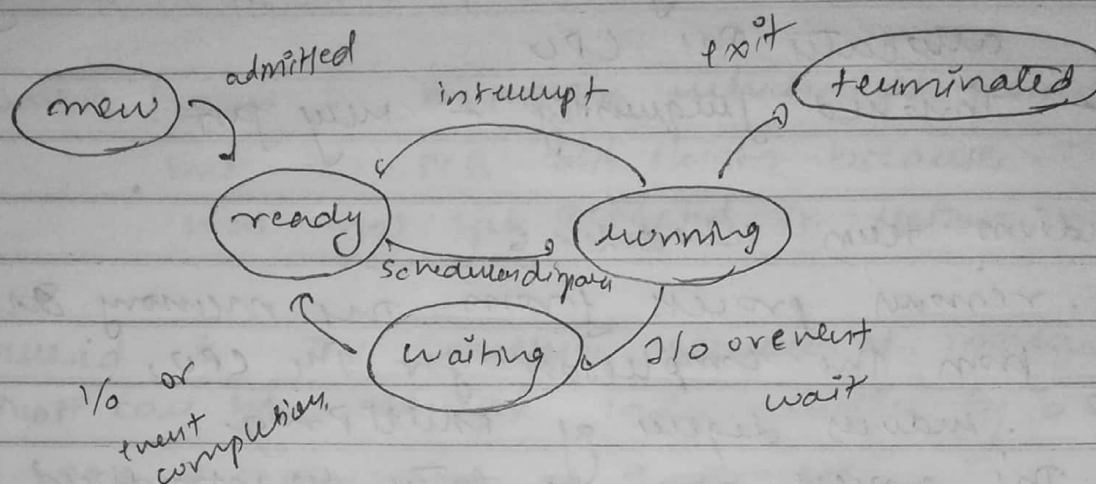
- Best avg. response time.
- Client server architecture.
- Kind SJF implementation.

Dis Adv :-

- longer process may starve.
- depends mainly on time quantum.
- no priority.

Program $\rightarrow$ Passive

Process $\rightarrow$ active.



PCB - Process Control block.

- Process scheduling is selecting one process for execution out of all the ready processes.
- ready queue $\rightarrow$ linked list of PCBs
- List of processes waiting for a particular I/O device is called a device queue. Each device has its own device queue.
- A process migrates among the various scheduling queues throughout its lifetime.

Three types of scheduling:

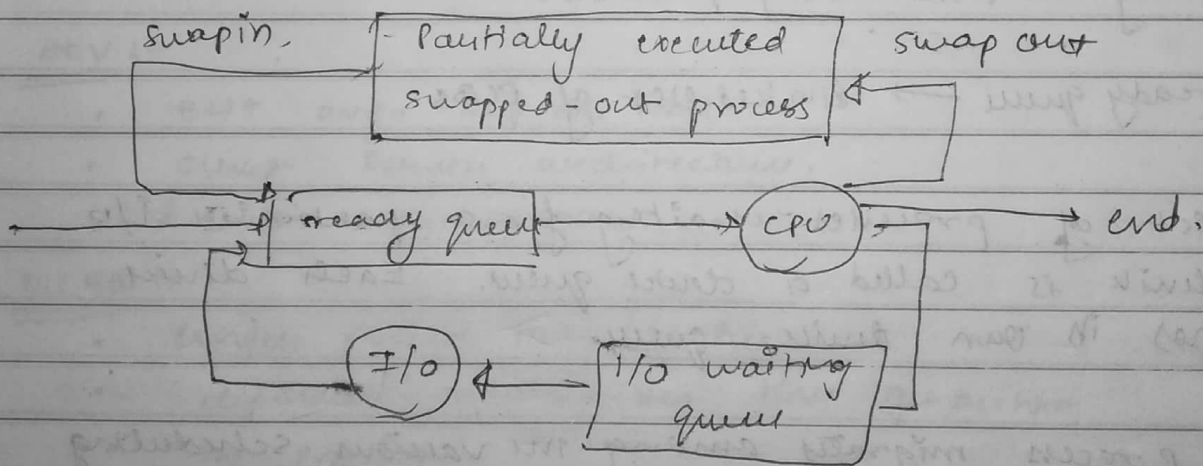
1. Job scheduler / long term scheduler:
 - selects processes from those submitted by the user and loads them into the memory.
 - controls degree of multiprogramming $\rightarrow$ no. of processes in memory
 - less frequently invoked.

20 Short term scheduler (CPU S-)

- selects one of the process in memory & allocates the CPU
- Invoked frequently & very fast.

30 Medium term scheduler

- removes process from the memory & from the completion for the CPU, makes degree of MultiProg
- The process can be later be introduced in the memory and their execution can be resumed. This is called swapping.



- context switching is done to switch b/w processes.
- Fast on RISC processors.
- C.S time is pure overhead time.

Process creation is one process creating another process.

- **Zombie Process** : A process which has terminated but its PCB still exists because its parent has not yet accepted its return value.
- **Thread** is the smallest sequence of instructions that can be managed independently by a scheduler.
 - It's a component of a process.
 - Multiple threads can appear exist within a single process, executing concurrently sharing resources such as memory.
 - Threads of a process share its instructions & its context (variables values).

Differences b/w Processes

- Independent
- Carry considerably more state information than Threads.
- Separate address
- Processes interact only through system-provided inter-process communication mechanisms.
- Context switch b/w processes is slower.

Threads

- exist as parts of a process.
- share process state as well as memory and other resources.
- share their address space.
- faster b/w threads in same process.

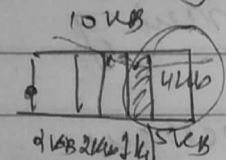
Adv of Multi threaded

- responsiveness
- faster execution
- better resource utilization
- easy communication
- parallelizable.

contiguous & non contiguous

fixed size partitioning

variable size partitioning



wasted (Internal Fragmentation)

2³ KB
8 KB

2⁷ MB
16 MB

2²⁰
2¹ 2¹⁹
(11)

Base address in the main mem.

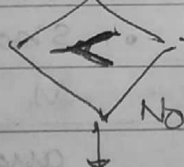
First Fit, Best Fit & Worst Fit

limit reg

relocation reg

CPU

logical address



Yes

No

trap



Physical memory

main mem.

→ when S.M is divided into partitions it is called page. & when M.M is divided into partitions they are called Frames

signal == V()
wait == P()

Producer consumer problem :-

$S = 1$; takes care of critical section.
 $E = n$; empty slots
 $F = 0$; Full slots.

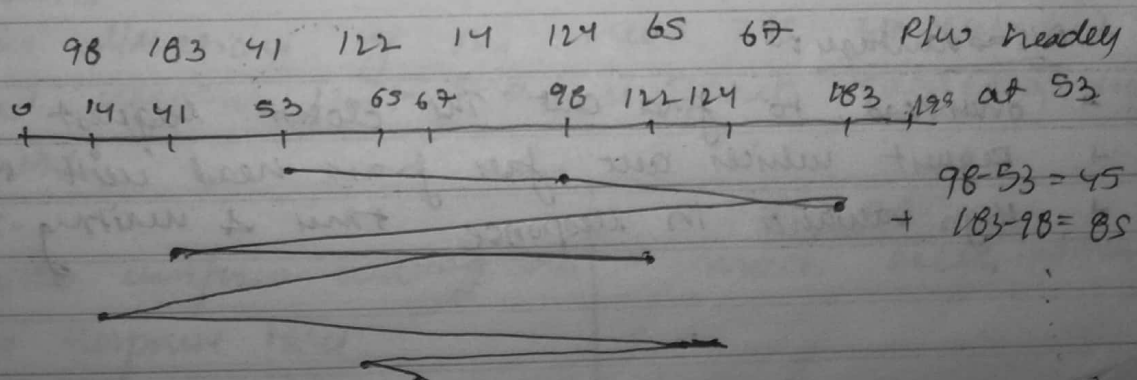
DISK Scheduling :-

Seek time : time required to move the R/W head on the desired track.

Disk Scheduling : In case of multiple I/O request, disk scheduling algo must decide which request must be executed first.

FCFS :

- Simplest disk scheduling algo.
- Request are entertained in order they are coming.



Advantages :

- Easy to understand & implement.
- no starvation (may suffer from convoy effect).
- can be used with less load.

Disadv :

- Requires more seek time.
- More waiting time & response time
- Inefficient.

SSTF (Shortest seek time first)

- services the request which is closest to the current position of R/W head.
- Tie is broken in the direction of head movement.

Advantages :-

- very efficient in seek moves.
- less average response time & waiting time.
- Increased throughput.

Disadvantages :-

- overhead to find out the closest request.
- Request which are far from head will starve.
- High variance in response time & waiting time.

SCAN :- Head starts at one end of the disk and moves towards the other end, servicing requests in between and reaches other end. and then direction of the head is reversed and process continues. Head continuously scans back and forth the disk.
→ sometimes called elevator algo.

Advantage:-

- simple easy to understand & implement
- no starvation (bounded waiting)
- low variance and average waiting time.

Disadvantage:-

- long waiting time for locations just visited by head.
- unnecessarily move till the end of the disk even if there is no request.

C-SCAN:- (Circular scan):

Head starts from one end of the disk and moves towards the other end, servicing requests in between, and reaches other end and then direction of the head is reversed & head reaches first end without satisfying any request.

Adv:-

- Provides uniform waiting time
- Better response time

dis adv:-

→ more seek moments

LOOK:- It is same as scan algo but instead of going till last track we go till last request and then change direction.

Advantage:-

- Better performance compared to scan
- should be used in case of less load.

Dis adv:-

- overhead to find the last request
- should not be used in case of more load.

C-LOOK:- It takes the advantage of both C-scan & look algo.

- will satisfy request only in one direction
- will go till last request and return but not till last track.

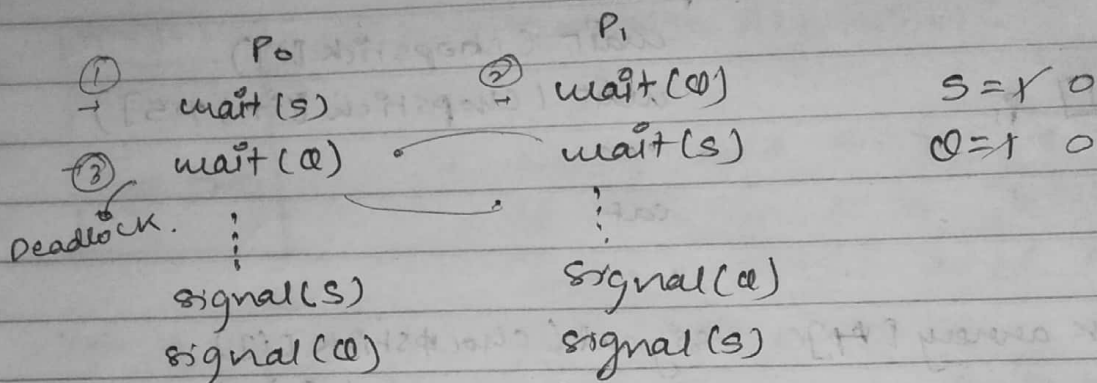
Advantage:-

- more uniform waiting time compared to look.
- more efficient (less moves) compared to C-scan

dis adv:-

- more overhead in cal.
- should not be used in case of more load.

Deadlock & Starvation



→ Improper use of semaphores can cause deadlock & starvation.

Bounded Buffer problems.

Place where you can store items.

→ There are n buffers where each & every buffer can store 1 item.

mutex: mutual exclusion variable.

initial value mutex = 1

empty = n

$$f_{\text{net}} = 0.$$

repeat

repeat

wait (full)

wait (mutes)

Produce an item in next

remove all items from
buffer to nextC

wait (empty)

Signal (mux)

$$\text{mit} + (\text{mit } x)$$

Signal (empty)

Signal (mutex)

signal (~~temp~~ full)

Consume the item in next c

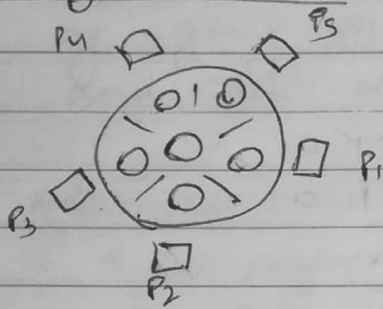
Unt⁹ falsch

until false

Producers

consumer

Dining Philosophers



repeat

wait (chopstick [i])

wait (chopstick [(i+1) % 5])

...

eat

...

var chopstick array [4] of semaphore

all 5 locations are initialized with 1.

signal (chopstick [i])

signal (chopstick [(i+1) % 5])

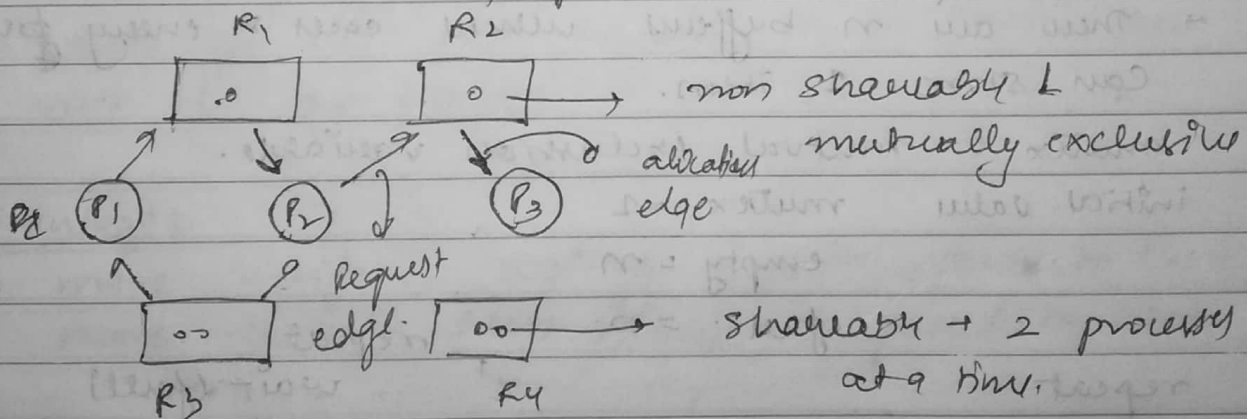
...

think

...

until false

Resource allocation graph



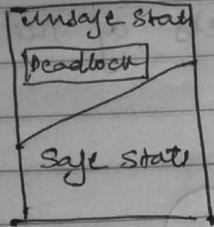
no. of dots $\rightarrow$ no. of process the resource

can be shared with

no cycle $\rightarrow$ no deadlock

cycle $\rightarrow$ may or may not occur

Deadlock avoidance :-



Banker's Algorithm :-

1. Work and Finish be two vectors of size $m \times n$. Initialize Work with available and $Finish[i] = 0$

$m \Rightarrow$ no. of resources

$n \Rightarrow$ no. of processes.

for $i = 1$ to n .

2. Find an i such that both

i) $Finish[i] = False$

ii) $Need_i \leq Work$

3. if no such i exists then goto step 4.
 $Work = Work + Allocation_i$

$Finish[i] = True$

goto step 2

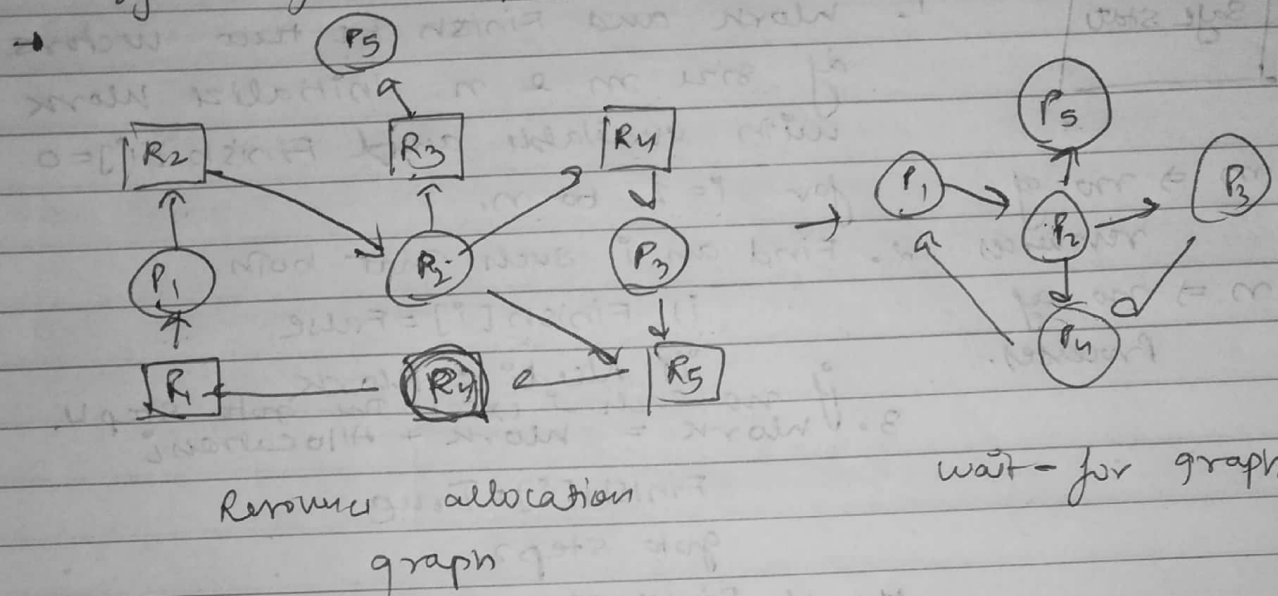
4. if $Finish[i] = True$ for all i then the system is in safe state.

Resource request Algorithm :-

1. if $request_i \leq need_i$ then goto step 2, else error
2. if $request_i \leq available_i$ then goto step 3, otherwise raise error wait until available.
3. $available_i = available_i - request_i$
 $need_i = need_i - request_i$
 $allocation_i = allocation_i + request_i$

Deadlock detection:

- whenever deadlock occurs the progress of the system gets suspended.

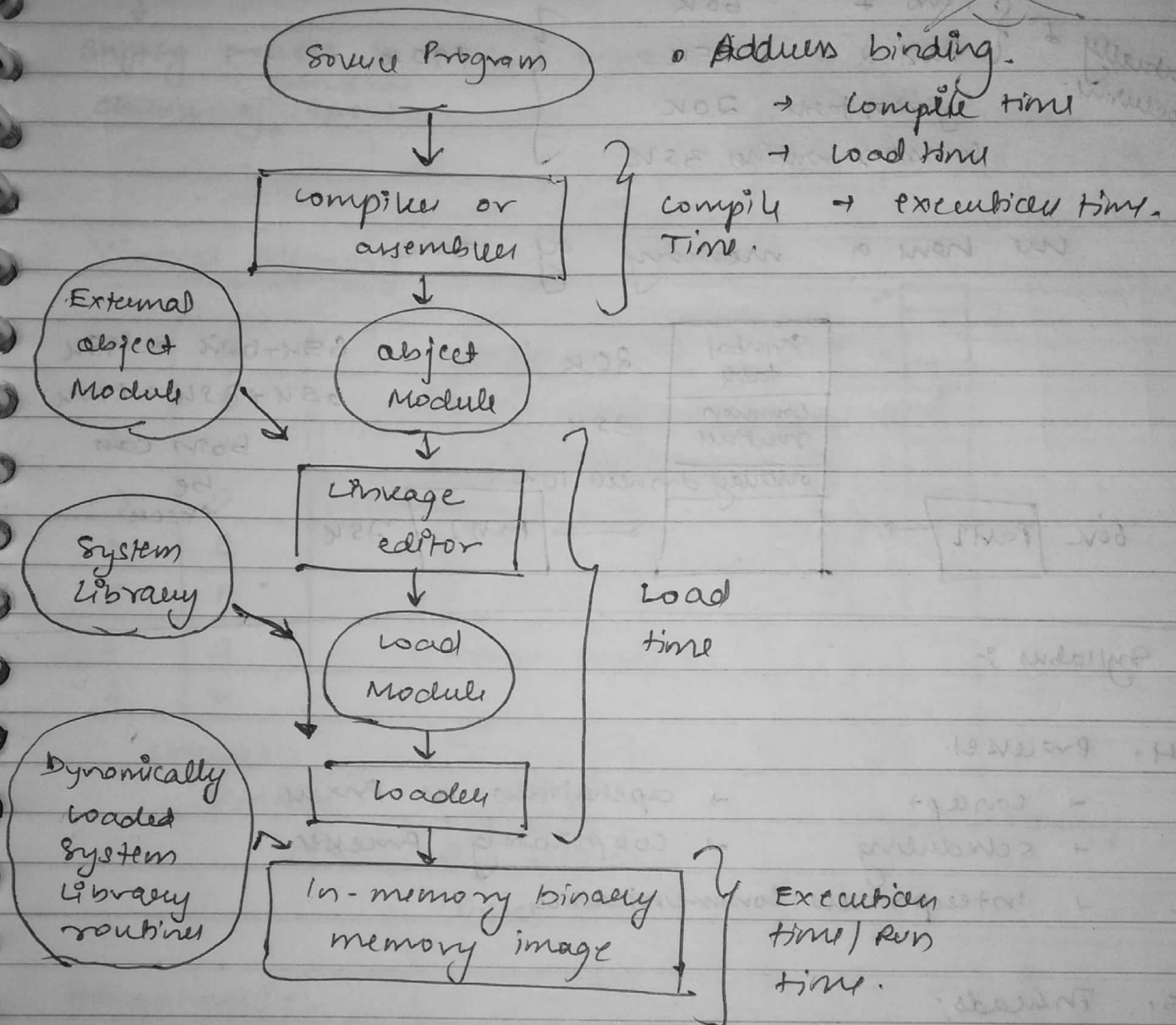


Deadlock recovery:

- Process termination.
 - Abort all deadlocked processes
 - Abort one process at a time until the deadlock cycle is eliminated.
- Resource preemption.
 - selecting a victim
 - Rollback
 - starvation.

should be done intelligently so that no process has to go through starvation.

Memory management :-



Multistep processing of a user program

- Dynamic loading :-** load the main modules first and the other whenever the program calls for other modules load them at that time. in a dynamic manner.

Stub.

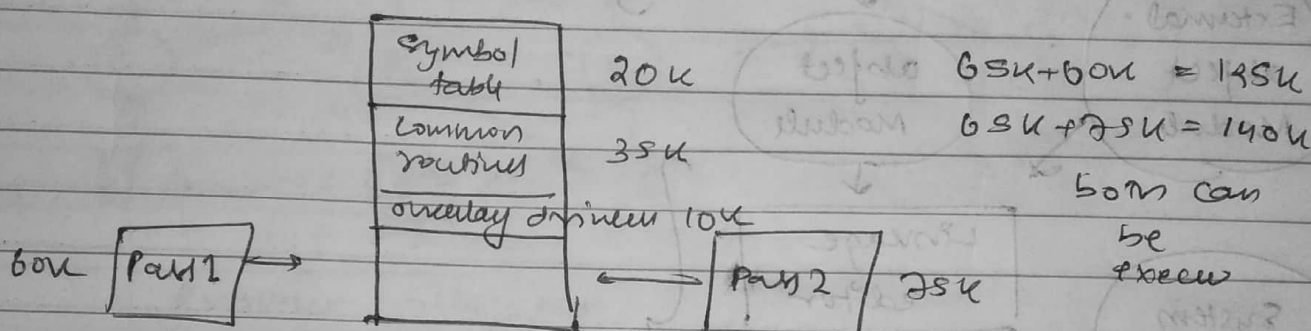
→ Overlays

mutually exclusive.

Part 1	60K
Part 2	75K
Symbol table	20K
Common routines	35K

190K

we have a memory of 150K



Syllabus :-

4. Processes.

- concept
- scheduling
- interprocess communication.
- operations on process
- cooperating processes.

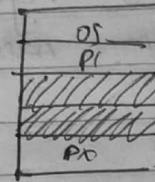
5. Threads.

- overview
- solutions 2 Thread
- Multithreaded Model
- Linux Thread.

6. CPU scheduling

Compaction & Coalescing :-

↓
shifting processes to make a bigger
continuous
chunk of space.



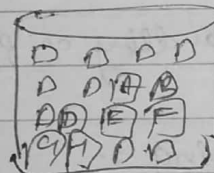
Virtual Memory :-

0	A
1	B
2	C
3	D
4	E
5	F
6	G
7	H

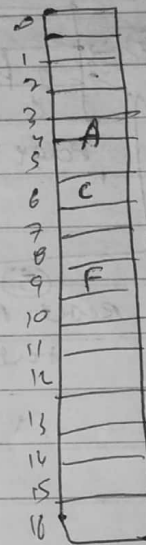
Logical
Memory

Frame no.	valid/invalid Bit
0	V
1	V
2	V
3	V
4	V
5	V
6	V
7	V

Page table



Disk



Physical Memory

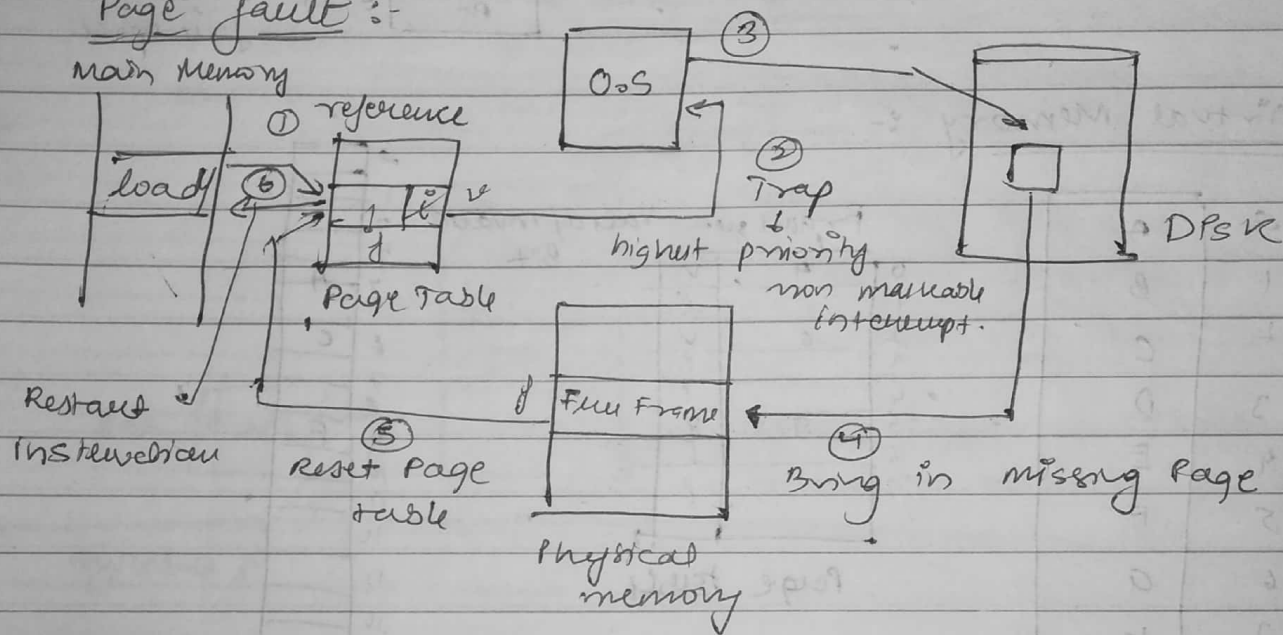
Advantages :-

- * Page will be brought into memory only when it is needed.
- Less I/O needed
- Less Memory needed
- Faster response
- More users

↓
• Lazy swapping
• Demand paging

- * Page is needed $\Rightarrow$ Reference to it
- Invalid reference $\Rightarrow$ Abort
- not in memory $\Rightarrow$ Bring to memory

Page fault :-



Steps in handling a page fault.

Performance of Demand Paging :-

- * $0 \leq \text{Page fault probability}(P) \leq 1$
- * Effective access time (EAT)

$$\text{EAT} = (1-P) * \text{memory access} + P * \text{page fault overhead}$$

- swap page out ✓
- swap page in
- Restart overhead.

Optimal page Replacement algo:-

- Replace the page that will not be used for longest period of time.
- Difficult to implement.

eg:-

	1	2	3	4	2	1	5	6	2	1	2	3	7	6	3	2	1	2	3	6
→	1	1	1	1	1	1	1	1					7					1		
→		2	2	2			2	2					2					2		
→			3	3			3	3					3					3		
→				4			5	6					6					6		
											↑ ↑ ↑ ↑					↑ ↑ ↑		↑ ↑ ↑		

8 miss =
Page faults.

LRU (Least recently used). use a stack.

	1	2	3	4	2	1	5	6	2	1	2	3	7	6	3	2	1	2	3	6
→	1	2	3	4	2	1	5	6	2	1	2	3	7	6	3	2	1	2	3	6
→		1	2	3	4	2	1	5	6	2	1	2	3	7	6	3	2	1	2	3
→			1	2	3	4	2	1	5	6	6	1	7	3	8	6	3	3	1	2
→				1	1	3	4	2	1	5	5	6	1	2	2	7	6	6	6	1
																↑ ↑		↑ ↑		↑ ↑

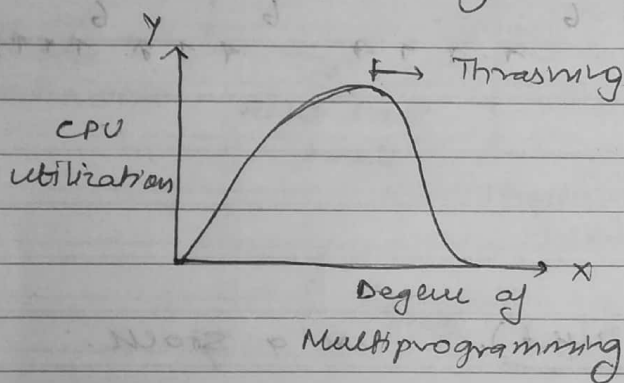
10 miss.

Another way counting.

8
13
14
14
10
81

Thrashing :-

- if we decrease the degree of multiprogramming then frames allocated to each program decreases.
- because of page replacement ~~it~~ will be required more and the page fault will increase as frames are less.
- the system will be busy not for execution of the program but for paging. ~~that~~ This situation is called thrashing.



Allocation Algorithm :-

$$P_1 = 10 \text{ pages}$$

$$P_2 = 127 \text{ pages}$$

$$\text{Free frames} = 62$$

$$\frac{10 \times 62}{10 + 127} \approx 4$$

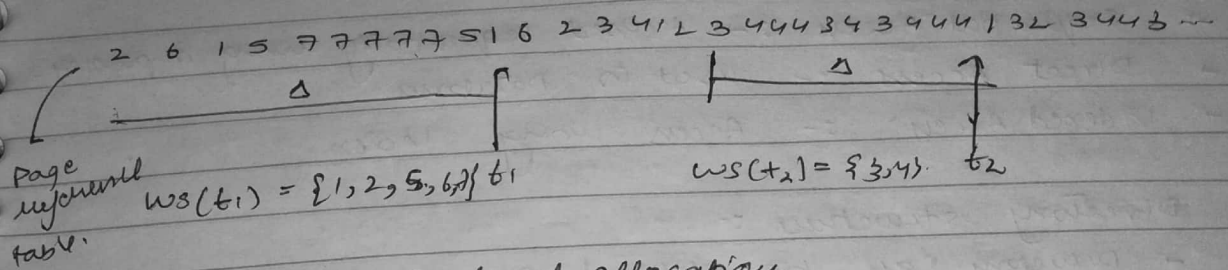
$$\frac{127 \times 62}{10 + 127} \approx 57$$

frames to P_1

giving frames to P_2

TLB $\rightarrow$ translation look aside buffer.

working set model.



Global allocation & local allocation.

when a process needs a page which is not in the memory, it can bring in the new page and allocate it a frame from the set of all frames, even if that frame is currently allocated to some other process; i.e., one process can take a frame from another.

When a process needs a page which is not in the memory, it can bring in the new page & allocate it a frame from its own set of allocated frames only.

ADV: Page fault rather than pages in memory only affected by that process only.

Truncate: It is simply deleting the file except deleting attributes. The file is not completely deleted although the information stored inside the file get replaced.

Delete: - will delete data, attributes the space allocated.

File Access methods :-

- Sequential Access :- Accessed sequentially using a pointer.
- Direct Access :- used in Databases
- Indexed Access :- Access using Index.

Directory structure :-

- Directory can be defined as the listing of the related files on the disk. The directory may store some or the entire file attributes.
- Directory can be viewed as a file which contains the metadata of the bunch of files.
- Single level directory :-
 - Two level directory :-
 - we can create a separate directory for each user. There is one master directory which contains separate directories dedicated to each user. For each user, there is a different directory present at the second level, level containing group of user's file.
 - Tree structured directory :-
 - any directory entry can be either be a file or sub directory.

Acyclic- Graph structured Directories :-

- Tree-SD
- This system doesn't allow the same file to exist in multiple directories therefore sharing is major concern in it. We can make provide sharing by making the directory an acyclic graph.

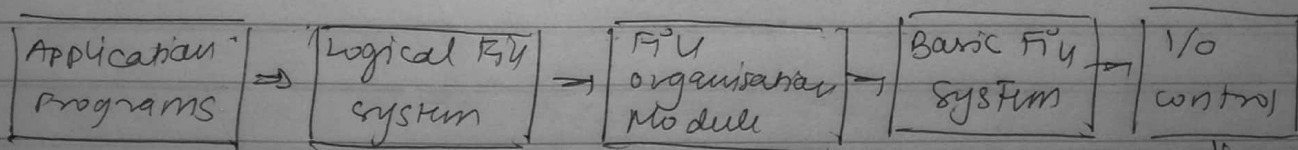
If a file gets deleted in acyclic graph structured directory system then.

1. In the case of soft link, the file just gets deleted & we are left with a dangling pointer.
2. In case of hard link, the actual file will be deleted only if all the references to it gets deleted.

FILE SYSTEMS :-

- Responsible for file management.
- Issues which it takes care of
 - File Structure. → maintain optimal file structures
 - Recovering free space :-
 - disk space assignment to the files.
 - tracking data locations.

File system structure :-



Master boot Record (MBR)

- Master boot record is the information present in the first sector of any hard disk.
- It contains the information regarding how and where the OS is located in the hard disk so that it can be booted in the RAM.
- Also called Master Partition table.
- It also includes a program which reads the boot sector record of the partition that contains the O.S.

On disk data structures:-

- Boot control block
 - UNIX → boot block
 - NTFS → partition boot sector.
- Volume control block
- Directory structure
- File control block.

I/O devices:-

PCI bus / IO bus

↓
Peripheral Component interconnect.